

```
"""
```

```
Cooper River Trading Co. –  
Appraze Auth Module
```

```
-----  
-----
```

```
Talks to the Apps Script Web App  
backend (see AppsScript_Code.gs)  
for  
tester signup/login. No Google  
service account or API key  
involved –  
just a token-protected HTTP  
endpoint hitting a "Users" tab on  
the  
existing Google Sheet.
```

```
Passwords are SHA-256 hashed  
client-side before ever leaving  
the app;  
the Sheet only ever stores the  
hash, never plaintext.
```

```
"""
```

```
import hashlib
from dataclasses import dataclass

import requests
import streamlit as st

@dataclass
class AuthResult:
    success: bool
    display_name: str = ""
    is_admin: bool = False
    is_paid: bool = False
    username: str = ""
    error: str = ""

def _hash_password(password: str)
-> str:
    return
    hashlib.sha256(password.encode("u
tf-8")).hexdigest()

def _apps_script_url() -> str:
    url =
```

```
st.secrets.get("APPS_SCRIPT_URL")
    if not url:
        raise
RuntimeError("APPS_SCRIPT_URL is
not set in Streamlit secrets.")
    return url
```

```
def _token() -> str:
    token =
st.secrets.get("APPS_SCRIPT_TOKEN
")
    if not token:
        raise
RuntimeError("APPS_SCRIPT_TOKEN
is not set in Streamlit
secrets.")
    return token
```

```
def signup(username: str,
password: str, display_name: str
= "", admin_code: str = "") ->
AuthResult:
    try:
        resp = requests.get(
```

```

        _apps_script_url(),
        params={
            "token":
_token(),
            "action":
"signup",
            "username":
username,
            "password_hash":
_hash_password(password),
            "display_name":
display_name or username,
            "admin_code":
admin_code,
        },
        timeout=15,
    )
    resp.raise_for_status()
    data = resp.json()
    if data.get("success"):
        return
    AuthResult(True,
data.get("display_name",
username), data.get("is_admin",
False), data.get("is_paid",
False),

```

```
username=data.get("username",
username.lower()))
        return AuthResult(False,
error=data.get("error", "signup
failed"))
    except Exception as e:
        return AuthResult(False,
error=f"connection error: {e}")
```

```
def login(username: str,
password: str) -> AuthResult:
    username_l =
username.strip().lower()
    password_hash =
_hash_password(password)
```

```
# 1. Check pre-provisioned
static accounts in secrets first
– no network
```

```
#    call needed, works even
if the Apps Script/Sheet side is
having issues.
```

```
static_users =
st.secrets.get("users", {})
if username_l in
```

```

static_users:
    rec =
static_users[username_l]
    if password_hash ==
rec.get("password_hash"):
        return AuthResult(
            True,

rec.get("display_name",
username_l),

bool(rec.get("is_admin", False)),

bool(rec.get("is_paid", True)),
# static accounts default to full
access

username=username_l,
        )
        return AuthResult(False,
error="incorrect password")

# 2. Fall back to the Apps
Script / Google Sheet backend for
anyone who
#     signs up fresh through

```

the normal Sign Up form (future customers).

```
    try:
        resp = requests.get(
            _apps_script_url(),
            params={
                "token":
_token(),
                "action":
"login",
                "username":
username,
                "password_hash":
password_hash,
            },
            timeout=15,
        )
        resp.raise_for_status()
        data = resp.json()
        if data.get("success"):
            return
    AuthResult(True,
data.get("display_name",
username), data.get("is_admin",
False), data.get("is_paid",
False),
```

```

username=data.get("username",
username_1))
        return AuthResult(False,
error=data.get("error", "login
failed"))
    except Exception as e:
        return AuthResult(False,
error=f"connection error: {e}")

def mark_paid(username: str) ->
bool:
    """Called once a Stripe
Checkout Session is verified as
paid – persists it
    so the person doesn't have to
pay again on their next login.
    Static
    secrets-based accounts are
already full-access, so this is a
no-op for them."""
    if username.strip().lower()
in st.secrets.get("users", {}):
        return True
    try:
        resp = requests.get(

```



```

        _apps_script_url(),
        params={"token":
_token(), "action": "set_paid",
"username": username},
        timeout=15,
    )
    resp.raise_for_status()
    return
bool(resp.json().get("success"))
except Exception:
    return False

```

```

def render_login_gate() -> bool:
    """
    Renders a login/signup form.
    Returns True if the current
    session is
        authenticated (and sets
    st.session_state.user_display_name / user_is_admin),
        False otherwise – caller
    should st.stop() when this
    returns False.
    """
    if

```

```
st.session_state.get("authenticated"):
    return True

    st.markdown("## 🏦 Appraze")
    st.caption("Cooper River
Trading Co. – beta access")

    tab_login, tab_signup =
st.tabs(["Log In", "Sign Up"])

    with tab_login:
        with
st.form("login_form"):
            u =
st.text_input("Username",
key="login_username")
            p =
st.text_input("Password",
type="password",
key="login_password")
            submitted =
st.form_submit_button("Log In",
use_container_width=True)
            if submitted:
                if not u or not
```

```
p:

st.warning("Enter both a username
and password.")
        else:
            result =
login(u, p)
            if
result.success:

st.session_state.authenticated =
True

st.session_state.user_display_name = result.display_name

st.session_state.user_is_admin =
result.is_admin

st.session_state.user_is_paid =
result.is_paid

st.session_state.username =
result.username

st.rerun()
```

```

else:

st.error(result.error)

    with tab_signup:
        st.caption("Pick your own
username and password – no admin
needed to set you up.")
        with
st.form("signup_form"):
            new_display =
st.text_input("Your name",
key="signup_display")
            new_u =
st.text_input("Choose a
username", key="signup_username")
            new_p =
st.text_input("Choose a
password", type="password",
key="signup_password")
            new_p2 =
st.text_input("Confirm password",
type="password",
key="signup_password2")
            admin_code =
st.text_input(

```

```
        "Admin invite  
code (leave blank unless you have  
one)",  
        type="password",  
  
key="signup_admin_code",  
    )  
    submitted =  
st.form_submit_button("Create  
Account",  
use_container_width=True)  
    if submitted:  
        if not new_u or  
not new_p:  
  
st.warning("Username and password  
are both required.")  
        elif new_p !=  
new_p2:  
  
st.warning("Passwords don't  
match.")  
        elif len(new_p) <  
4:  
  
st.warning("Password should be at
```

```
least 4 characters.")
    else:
        result =
signup(new_u, new_p, new_display,
admin_code)
        if
result.success:

st.session_state.authenticated =
True

st.session_state.user_display_name = result.display_name

st.session_state.user_is_admin =
result.is_admin

st.session_state.user_is_paid =
result.is_paid

st.session_state.username =
result.username

st.success(f"Welcome,
{result.display_name}!")
```

```
st.rerun()  
                else:  
st.error(result.error)  
        return False
```